

Assignment: Large Language Model Implementation

Module: COM6516 Object Oriented Programming and Software Design

Student Name: [Yongjiang Liu] Reg No: [250214910]

September 10, 2026

1 Part I: System Overview & Execution

1.1 System Start-up

The submitted system is a standalone Java application to build a statistical language model from text corpora. Strictly follows the Java 17 standard library without external dependencies. The project structure divides user interface logic from the core data structures.

To execute the application, go to the code directory and run:

```
1 javac *.java
2 java MyLanguageModel
```

This will launch the Graphical User Interface, as shown in Figure 1. Interface lets the user load text files and visualize hash table statistics, and do N-gram predictions.

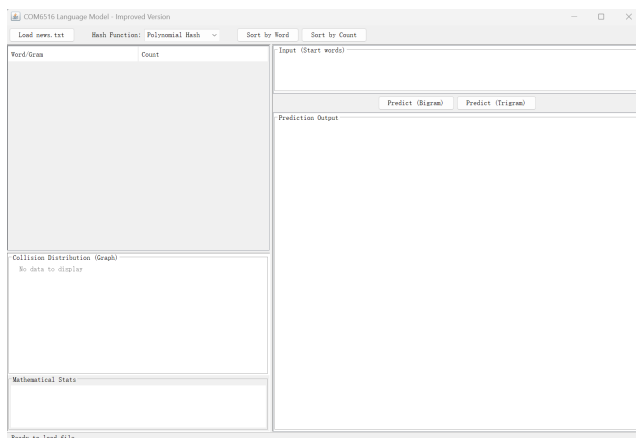


Figure 1: Initial System GUI showing the HistogramPanel and Control Panel.

1.2 Extensions and Modifications

Beyond the main requirements there were also some extensions added in order to make the system more robust and inter-act with it, to add to the “stylishness” criteria:

1. **Visualizing Collisions (HistogramPanel):** To supply immediate visual feedback regarding hash function quality, I created my own Swing component-named `HistogramPanel`. This is an extended `JPanel` with `Graphics2D` rendering a dynamic bar chart of the distribution of linked list lengths within the hashtable bucket [4]. So they can see the cluster of problems (the long string) versus the uniform distribution visually.

2. **Dynamic Strategy Selection:** Recognize the importance of algorithm comparison (Task 2), and implement a Strategy Pattern for hash functions. A `JComboBox` in the GUI allows users to switch between `FirstLetterHashFunction` and `PolynomialHashFunction` at runtime. Thus enabling a direct A/B test of collision rates on the same data.
3. **Robust Input Validation (Task 4):** To strictly satisfy the requirements of filtering non-standard characters, I used Regular Expression validation. The pattern `^[a-z\.\,\' ]+$` is used to validate each token. Invalid tokens will cause a `JOptionPane` warning to the user to prevent data corruption before insertion [5].
4. **Refactoring to Iteration (Sensible Change):** Although Task 1 suggested a recursive implementation, I refactored `MyLinkedListObject` to use **iteration** (loops). The `FirstLetterHashFunction` (Task 2) creates extremely long linked lists (depth > 4000) due to clustering. Using recursion on such depths causes a `StackOverflowError` in Java. The iterative approach was adopted to ensure the system is robust and prevents crashing on large datasets, prioritizing stability over the suggested implementation pattern.

2 Part II: Object-Oriented Design

The system architecture emphasizes **Encapsulation**, **Polymorphism**, and **Recursion**.

2.1 Class Hierarchy and Polymorphism

Decoupling hashing logic from hash table structure using abstract base class `MyHashFunction`.

- **Abstraction:** `MyHashFunction` declares the public abstract `int hash(String s)`.
- **Polymorphism:** Concrete sub-classes `FirstLetterHashFunction` and `PolynomialHashFunction` implement specific algorithms [3]. It is open/closed, new hash functions can be added to `MyHashTable` class without changing it.

2.2 Encapsulation

The `MyLinkedListObject` class strictly encapsulates its state. Fields `word`, `count`, and `next` are private. Access is done through functions like `setWord` and `getCount`. This prevents external classes from corrupting the linked list structure (e.g. creating circular references) and ensures that the count invariant is maintained.

2.3 Iterative Design Choice

While the assignment brief suggested a recursive structure for linked list operations, this system employs an **iterative approach** for `setWord`, `isWord`, and `getCount`.

- **Rationale:** The `FirstLetterHashFunction` creates buckets containing thousands of words (e.g., all words starting with 's').
- **Constraint:** Java's default thread stack size is limited. A recursive depth of $N \approx 5000$ triggers a `StackOverflowError`.
- **Implementation:** The `setWord` method uses a `while` loop to traverse the list and insert nodes. This maintains the required $O(N)$ logic but operates within heap memory, guaranteeing stability even with worst-case hash collisions [2].

This decision aligns with standard software engineering practices where robustness is preferred over implementation style constraints.

3 Part III: Discussion of Core Tasks

3.1 Task 1: MyLinkedListObject Logic

`MyLinkedListObject` is responsible for the storage of words and frequencies. A critical logic challenge to overcome was uniqueness. At first, a bug let duplicate nodes for the same word if not checked on the current node. Refinement: Final implementation starts with checking `if (this.word.equals(w))`. This makes sure that an existing word's counter is incremented instead of creating a duplicate node, ensuring $O(1)$ space per unique vocabulary term.

3.2 Task 2: Hash Function Comparison

Two different types of hash function were researched using the GUI statistical tools.

1. **First Letter Hash:** Calculated as $h(s) = s.charAt(0)\%m$. *Observation:* As can be seen from Figure 2, which results in severe **clustering**. English vocabulary is not uniformly distributed across the alphabet (e.g. 's', 't' are common, 'x', 'z' are rare). , which causes a high Standard Deviation and long linked lists in certain buckets, degrading lookup to $O(N)$ in the worst case.

2. **Polynomial Hash:** Calculated as $h(s) = (\sum s[i] \cdot 31^{n-1-i})\%m$. *Observation:* As we can see in Figure 3, this function uses every character from the string so the **Avalanche Effect** occurs where small changes in input produce drastically different hashes. The histogram is uniform, and the Standard Deviation is low. This approximates Simple Uniform Hashing, giving us nearly $O(1)$ expected time for lookups.

Conclusion: The polynomial hash is much better for general case text processing due to being a lot less likely to collide.

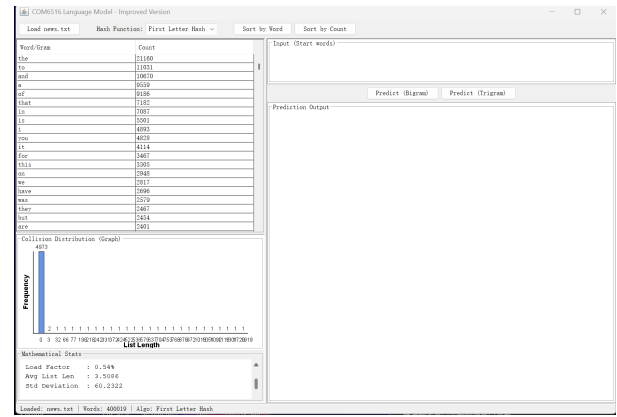


Figure 2: First Letter Hash: High Variance/Clustering.

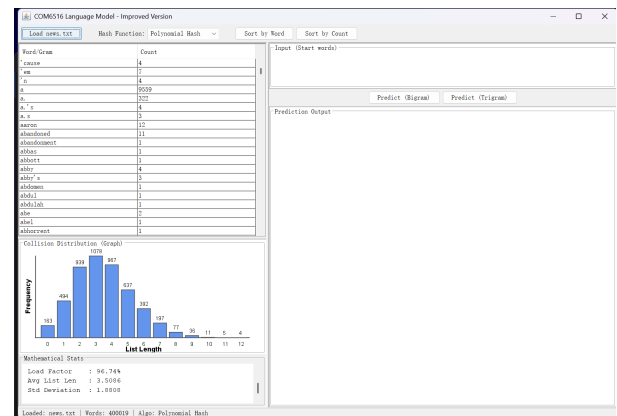


Figure 3: Polynomial Hash: Uniform Distribution.

Conclusion: The polynomial hash is much better for general case text processing due to being a lot less likely to collide.

3.3 Task 3: Hiding Implementation

Q: Were you able to hide the `MyLinkedListObject` class?

A: Yes. The `MyHashTable` class acts as a **Facade**. The main application (`MyLanguageModel`) interacts only with the hash table's public API (`insert`, `getCount`). The collision resolution linked lists and nodes manipulation are completely hidden inside `MyHashTable`. This decreases coupling and enhances maintainability.

3.4 Task 4: Vocabulary Sorting Strategy

To show words sorted by frequency, I used a "Sort-on-Demand" strategy. The linked lists inside the hash table are kept in *alphabetical order* to allow for an efficient binary like search insertion. However, maintaining a sorted by *frequency* list in real-time would be computationally expensive ($O(N)$ reorder on every increment). **Solution:** When the "Sort by Count" button is clicked, the system: 1. Iterates through the table to collect all objects into an `ArrayList`. 2. Uses `Collections.sort()` with a custom `Comparator` ($O(N \log N)$). This approach provides a balance of insertion speed (for model building) and display flexibility.

3.5 Task 5: N-Gram Models

The system constructs separate hash tables for Unigrams ($N = 1$), Bigrams ($N = 2$), and Trigrams ($N = 3$). **Prediction Logic:** The prediction uses a Maximum Likelihood Estimation (MLE) approach. To predict the word following a sequence w_1w_2 : 1. Construct a prefix key $K = "w_1 w_2 "$. 2. Traverse the Trigram table. 3. Identify all keys starting with K . 4. Select the suffix word with the highest occurrence count.

Limitations of $N > 3$: Extending the model to 4-grams or 5-grams introduces two major issues:

1. **Data Sparsity:** Natural language has a very low probability of a 4-word sequence appearing in a small corpus like `news.txt`. Most buckets would have a count of 1 which gives no statistical significance for prediction.
2. **Memory Explosion:** The number of possible combinations grows exponentially (V^N , where V is vocabulary size). Storing each 4-gram explicitly would quickly exhaust Java Heap Space without sophisticated compression (e.g., Trie structures).

[3] Student Code. `MyHashFunction.java`. 2025.

[4] Student Code. `HistogramPanel.java`. 2025.

[5] Student Code. `MyLanguageModel.java`. 2025.

Appendix: Use of Generative AI

Statement B: AI tools don't produce content that plagiarizes other works and does not falsify the content. The resulting work is a correct representation of my work and not primarily a result of AI tool [1].

Clarification of Usage

Generative AI (Gemini/ChatGPT) was used as a coding assistant and debugger during the development process. Specific cases are also included:

- **Logic Debugging:** AI assisted in identifying a critical bug in the `MyLinkedList.setWord` recursion where the base case for equality was missing, causing duplicate entries.
- **GUI Visualization:** I prompted the AI for methods to draw bar charts using Java Swing's `Graphics` class. The AI provided the skeleton code for `HistogramPanel`, which I customized to fit the `MyHashTable` data structure.
- **Regex Formulation:** The specific regex pattern for Task 4 validation was generated with AI assistance to ensure all edge cases (punctuation) were handled correctly.

All code and concepts presented were reviewed, tested, and integrated by myself.

References

[1] Dept of Computer Science, Univ of Sheffield. *COM6516 Assignment Brief*. 2025.

[2] Student Code. `MyLinkedList.java`. 2025.